

Somaiya Vidyavihar University
K J Somaiya School of Engineering
Department of Electronics and Telecommunication Engineering



Somaiya Vidyavihar University
K J Somaiya School of Engineering



Department of Electronics and Telecommunication Engineering

Batch: A2 Roll No.: 16010324044

Experiment / assignment / tutorial No. 5

Grade: AA / AB / BB / BC / CC / CD / DD

Signature of the Staff In-charge with date

for
10/9/26

Experiment No: 5

Title: Queries based on Joins, and Views

Objective: To be able to use SQL JOIN clauses to extract data from 2 (or more) tables, we need a relationship between certain columns in these tables.

Expected Outcome of Experiment:

CO 3 : Use SQL for Relational database creation, maintenance and query processing

CO 4 : Applying normalization to design database

Books/ Journals/ Websites referred:

1. Dr. P.S. Deshpande, SQL and PL/SQL for Oracle 10g. Black book, Dreamtech Press
2. www.db-book.com
3. Korth, Silberchatz, Sudarshan : "Database Systems Concept", 5th Edition , McGraw Hill
4. Elmasri and Navathe, "Fundamentals of database Systems", 4th Edition, PEARSON Education.

Resources used: Postgresql

Theory

Join is a combination of a Cartesian product followed by a selection process. A Join operation pairs two tuples from different relations, if and only if a given join condition is satisfied. Or JOINS are used to retrieve data from multiple tables. A JOIN is performed whenever two or more tables are joined in a SQL statement.

There are different types of Joins:

Department of Electronics and Telecommunication Engineering
DMSL –Sem- VI- Jan –April 2026

Department of Electronics and Telecommunication Engineering
DMSL –Sem- VI- Jan –April 2026

Somaia Vidyavihar University
K J Somaia School of Engineering
Department of Electronics and Telecommunication Engineering

Experiment No: 5

Title: Queries based on Joins, and Views

Objective: To be able to use SQL JOIN clauses to extract data from 2 (or more) tables, we need a relationship between certain columns in these tables.

Expected Outcome of Experiment:

CO 3 : Use SQL for Relational database creation, maintenance and query processing

CO 4 : Applying normalization to design database

Books/ Journals/ Websites referred:

1. Dr. P.S. Deshpande, SQL and PL/SQL for Oracle 10g.Black book, Dreamtech Press
2. www.db-book.com
3. Korth, Silberchatz, Sudarshan : “Database Systems Concept”, 5th Edition , McGraw Hill
4. Elmasri and Navathe,”Fundamentals of database Systems”, 4th Edition,PEARSON Education.

Resources used: Postgresql

Theory

Join is a combination of a Cartesian product followed by a selection process. A Join operation pairs two tuples from different relations, if and only if a given join condition is satisfied. Or JOINS are used to retrieve data from multiple tables. A JOIN is performed whenever two or more tables are joined in a SQL statement.

There are different types of Joins:

- The CROSS JOIN

Somaiya Vidyavihar University
K J Somaiya School of Engineering
Department of Electronics and Telecommunication Engineering

- The INNER JOIN
- The LEFT OUTER JOIN
- The RIGHT OUTER JOIN
- The FULL OUTER JOIN

A **CROSS JOIN** matches every row of the first table with every row of the second table. If the input tables have x and y columns, respectively, the resulting table will have x+y columns. Because CROSS JOINS have the potential to generate extremely large tables, care must be taken to use them only when appropriate.

Ex. SELECT EMP_ID, NAME, DEPT FROM COMPANY CROSS JOIN DEPARTMENT;

A **INNER JOIN** creates a new result table by combining column values of two tables (table1 and table2) based upon the join-predicate. The query compares each row of table1 with each row of table2 to find all pairs of rows, which satisfy the join-predicate. When the join-predicate is satisfied, column values for each matched pair of rows of table1 and table2 are combined into a result row.

Ex. SELECT EMP_ID, NAME, DEPT FROM COMPANY INNER JOIN DEPARTMENT
ON COMPANY.ID = DEPARTMENT.EMP_ID;

The **OUTER JOIN** is an extension of the INNER JOIN. SQL standard defines three types of OUTER JOINS: LEFT, RIGHT, and FULL and PostgreSQL supports all of these.

In case of **LEFT OUTER JOIN**, an inner join is performed first. Then, for each row in table T1 that does not satisfy the join condition with any row in table T2, a joined row is added with null values in columns of T2. Thus, the joined table always has at least one row for each row in T1.

Ex. SELECT EMP_ID, NAME, DEPT FROM COMPANY LEFT OUTER JOIN
DEPARTMENT ON COMPANY.ID = DEPARTMENT.EMP_ID;

The RIGHT OUTER JOIN

First, an inner join is performed. Then, for each row in table T2 that does not satisfy the join condition with any row in table T1, a joined row is added with null values in columns of T1. This is the converse of a left join; the result table will always have a row for each row in T2.

Ex. SELECT EMP_ID, NAME, DEPT FROM COMPANY RIGHT OUTER JOIN
DEPARTMENT ON COMPANY.ID = DEPARTMENT.EMP_ID;

The **FULL OUTER JOIN**

Somaiya Vidyavihar University
K J Somaiya School of Engineering
Department of Electronics and Telecommunication Engineering

First, an inner join is performed. Then, for each row in table T1 that does not satisfy the join condition with any row in table T2, a joined row is added with null values in columns of T2. In addition, for each row of T2 that does not satisfy the join condition with any row in T1, a joined row with null values in the columns of T1 is added.

```
SELECT EMP_ID, NAME, DEPT FROM COMPANY FULL OUTER JOIN
DEPARTMENT ON COMPANY.ID = DEPARTMENT.EMP_ID;
```

Problem Statement: Employee-Project Management System

A mid-sized IT company wants to analyze its workforce and project allocation data. The company maintains records of employees, departments, projects, and the assignments linking employees to projects. However, not every employee is currently assigned to a project, not every department has employees, and not every project has been assigned any employee yet. The management wants a unified reporting system that can answer different business questions — some requiring only matched records, some requiring all records regardless of matches, and one requiring every possible combination of two datasets for planning purposes.

You are asked to design the database schema and write SQL queries using CROSS JOIN, INNER JOIN, LEFT OUTER JOIN, RIGHT OUTER JOIN, and FULL OUTER JOIN to meet the following requirements.

Schema Design

1. Department

Column	Type	Description
dept_id	INT (PK)	Unique department identifier
dept_name	VARCHAR(50)	Name of the department

2. Employee

Column	Type	Description
emp_id	INT (PK)	Unique employee identifier
emp_name	VARCHAR(50)	Employee name
dept_id	INT (FK → Department.dept_id)	Department the employee belongs to

Somaiya Vidyavihar University
 K J Somaiya School of Engineering
Department of Electronics and Telecommunication Engineering

3. Project

Column	Type	Description
proj_id	INT (PK)	Unique project identifier
proj_name	VARCHAR(50)	Name of the project
budget	DECIMAL(10,2)	Project budget

4. Assignment

Column	Type	Description
assign_id	INT (PK)	Unique assignment identifier
emp_id	INT (FK → Employee.emp_id)	Employee assigned
proj_id	INT (FK → Project.proj_id)	Project assigned to
role	VARCHAR(50)	Role of employee in the project

Business Requirements (Map to Join Types)

a) INNER JOIN — HR wants a report of only those employees who are **currently assigned to a project**, showing employee name, project name, and their role. Employees without any assignment should not appear.

b) LEFT OUTER JOIN — Management wants a complete list of **all employees**, along with their project details if assigned — including employees who are **not currently working on any project** (project fields should show NULL for them).

c) RIGHT OUTER JOIN — The PMO (Project Management Office) wants a list of **all projects**, along with the employees working on them — including **projects that have not yet been assigned any employee** (employee fields should show NULL for them).

d) FULL OUTER JOIN — The company wants a consolidated report showing **every employee and every project relationship**, including employees with no project and projects with no employees, so gaps in resource allocation can be identified in a single view.

e) CROSS JOIN — For a hypothetical resource-planning exercise, the management wants to generate **every possible combination of employees and projects** (regardless of actual assignment), to simulate potential allocation scenarios and estimate workload distribution if every employee could theoretically be assigned to every project.

Somaia Vidyavihar University
K J Somaia School of Engineering
Department of Electronics and Telecommunication Engineering

Implementation Screenshots (Already allocated Problem Statement , Query (wrt. Table all variety of joins implementation as well as view)and Screenshots of Results):

Create Table:

```
CREATE TABLE Department (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50)  
);  
  
CREATE TABLE Employee (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    dept_id INT,  
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)  
);  
  
CREATE TABLE Project (  
    proj_id INT PRIMARY KEY,  
    proj_name VARCHAR(50),  
    budget DECIMAL(10,2)  
);  
  
CREATE TABLE Assignment (  
    assign_id INT PRIMARY KEY,  
    emp_id INT,  
    proj_id INT,  
    role VARCHAR(50),  
    FOREIGN KEY (emp_id) REFERENCES Employee(emp_id),  
    FOREIGN KEY (proj_id) REFERENCES Project(proj_id)  
);
```

Sample Data:

Somaiya Vidyavihar University
K J Somaiya School of Engineering
Department of Electronics and Telecommunication Engineering

INSERT INTO Department VALUES

```
(1, 'IT'),  
(2, 'HR'),  
(3, 'Finance'),  
(4, 'Marketing');
```

INSERT INTO Employee VALUES

```
(101, 'Amit', 1),  
(102, 'Priya', 1),  
(103, 'Rahul', 2),  
(104, 'Sneha', 3),  
(105, 'Vikas', 4);
```

INSERT INTO Project VALUES

```
(201, 'Website Development', 500000.00),  
(202, 'Mobile App', 350000.00),  
(203, 'Cloud Migration', 750000.00),  
(204, 'AI Research', 900000.00);
```

INSERT INTO Assignment VALUES

```
(1, 101, 201, 'Developer'),  
(2, 102, 202, 'Developer'),  
(3, 103, 201, 'Tester'),  
(4, 104, 203, 'Database Engineer');
```

Verify Table:

None

```
SELECT * FROM Department;
```

```
SELECT * FROM Employee;
```

```
SELECT * FROM Project;
```

```
SELECT * FROM Assignment;
```

Somaiya Vidyavihar University
K J Somaiya School of Engineering
Department of Electronics and Telecommunication Engineering

	assign_id [PK] integer	emp_id integer	proj_id integer	role character varying (50)
1	1	101	201	Developer
2	2	102	202	Developer
3	3	103	201	Tester
4	4	104	203	Database Engineer

Cross Join:

None

SELECT

e.emp_id,

e.emp_name,

p.proj_id,

p.proj_name

FROM Employee e

CROSS JOIN Project p;

Somaiya Vidyavihar University
 K J Somaiya School of Engineering
Department of Electronics and Telecommunication Engineering

	emp_id integer	emp_name character varying (50)	proj_id integer	proj_name character varying (50)
1	101	Amit	201	Website Development
2	101	Amit	202	Mobile App
3	101	Amit	203	Cloud Migration
4	101	Amit	204	AI Research
5	102	Priya	201	Website Development
6	102	Priya	202	Mobile App
7	102	Priya	203	Cloud Migration
8	102	Priya	204	AI Research
9	103	Rahul	201	Website Development
10	103	Rahul	202	Mobile App
11	103	Rahul	203	Cloud Migration
12	103	Rahul	204	AI Research
13	104	Sneha	201	Website Development
14	104	Sneha	202	Mobile App
15	104	Sneha	203	Cloud Migration
16	104	Sneha	204	AI Research
17	105	Vikas	201	Website Development
18	105	Vikas	202	Mobile App
19	105	Vikas	203	Cloud Migration
20	105	Vikas	204	AI Research

Inner Join:

None

SELECT

e.emp_name,

p.proj_name,

a.role

FROM Employee e

Somaiya Vidyavihar University
K J Somaiya School of Engineering
Department of Electronics and Telecommunication Engineering

INNER JOIN Assignment a

ON e.emp_id = a.emp_id

INNER JOIN Project p

ON a.proj_id = p.proj_id;

	emp_name character varying (50) 🔒	proj_name character varying (50) 🔒	role character varying (50) 🔒
1	Amit	Website Development	Developer
2	Priya	Mobile App	Developer
3	Rahul	Website Development	Tester
4	Sneha	Cloud Migration	Database Engineer

Left Outer Join:

None

SELECT

e.emp_id,

e.emp_name,

p.proj_name,

a.role

FROM Employee e

LEFT OUTER JOIN Assignment a

ON e.emp_id = a.emp_id

LEFT OUTER JOIN Project p

ON a.proj_id = p.proj_id;

Somaia Vidyavihar University
K J Somaia School of Engineering
Department of Electronics and Telecommunication Engineering

	emp_id integer	emp_name character varying (50)	proj_name character varying (50)	role character varying (50)
1	101	Amit	Website Development	Developer
2	102	Priya	Mobile App	Developer
3	103	Rahul	Website Development	Tester
4	104	Sneha	Cloud Migration	Database Engineer
5	105	Vikas	[null]	[null]

Right Outer Join:

None

```
SELECT
    p.proj_id,
    p.proj_name,
    e.emp_name,
    a.role
FROM Employee e
RIGHT OUTER JOIN Assignment a
    ON e.emp_id = a.emp_id
RIGHT OUTER JOIN Project p
    ON a.proj_id = p.proj_id;
```

- Equivalent:

None

```
SELECT
    p.proj_id,
    p.proj_name,
```

Somaia Vidyavihar University
K J Somaia School of Engineering
Department of Electronics and Telecommunication Engineering

```
e.emp_name,  
a.role  
FROM Project p  
LEFT JOIN Assignment a  
ON p.proj_id = a.proj_id  
LEFT JOIN Employee e  
ON a.emp_id = e.emp_id;
```

	proj_id integer 🔒	proj_name character varying (50) 🔒	emp_name character varying (50) 🔒	role character varying (50) 🔒
1	201	Website Development	Amit	Developer
2	202	Mobile App	Priya	Developer
3	201	Website Development	Rahul	Tester
4	203	Cloud Migration	Sneha	Database Engineer
5	204	AI Research	[null]	[null]

Full Outer Join:

None

SELECT

```
e.emp_id,  
e.emp_name,  
p.proj_id,  
p.proj_name,  
a.role
```

Somaia Vidyavihar University
K J Somaiya School of Engineering
Department of Electronics and Telecommunication Engineering

```
FROM Employee e

FULL OUTER JOIN Assignment a

    ON e.emp_id = a.emp_id

FULL OUTER JOIN Project p

    ON a.proj_id = p.proj_id;
```

	emp_id integer 🔒	emp_name character varying (50) 🔒	proj_id integer 🔒	proj_name character varying (50) 🔒	role character varying (50) 🔒
1	101	Amit	201	Website Development	Developer
2	102	Priya	202	Mobile App	Developer
3	103	Rahul	201	Website Development	Tester
4	104	Sneha	203	Cloud Migration	Database Engineer
5	105	Vikas	[null]	[null]	[null]
6	[null]	[null]	204	AI Research	[null]

Creating a View:

```
None

CREATE VIEW Employee_Project_View AS

SELECT

    e.emp_id,

    e.emp_name,

    p.proj_name,

    a.role

FROM Employee e
```

Somaia Vidyavihar University
 K J Somaia School of Engineering
Department of Electronics and Telecommunication Engineering

LEFT JOIN Assignment a

ON e.emp_id = a.emp_id

LEFT JOIN Project p

ON a.proj_id = p.proj_id;

None

SELECT * FROM Employee_Project_View;

	emp_id integer 🔒	emp_name character varying (50) 🔒	proj_name character varying (50) 🔒	role character varying (50) 🔒
1	101	Amit	Website Development	Developer
2	102	Priya	Mobile App	Developer
3	103	Rahul	Website Development	Tester
4	104	Sneha	Cloud Migration	Database Engineer
5	105	Vikas	[null]	[null]

Post Lab Questions:

1. What is a view?

a) A view is a special stored procedure executed when certain event occurs

b) A view is a virtual table which results of executing a pre-compiled query

c) A view is a database diagram

d) None of the Mentioned

2. What type of join is needed when you wish to include rows that do not have matching values?

Somaiya Vidyavihar University
K J Somaiya School of Engineering
Department of Electronics and Telecommunication Engineering

- A. Equi-join
- B. Natural join
- C. Outer join**
- D. All of the mentioned

Conclusion:

In this experiment, I successfully implemented SQL JOINS and Views were in PostgreSQL. Different joins were used to retrieve both matching and non-matching records from multiple tables, demonstrating their use in relational database query processing.

Signature of faculty in-charge